



Lecture 4

Architectural Patterns

Sharif University of Technology
Spring 2026

A Quick Review!



- Software architecture defines the **high-level** structure of a software system and how its major components **interact**.
- It establishes core design decisions, including system organization, communication patterns, and technology choices.
- Software architecture addresses non-functional requirements such as scalability, performance, security, and reliability.
- It provides a **shared blueprint** that guides development, deployment, and long-term evolution of the system.
- The goal of software architecture is to **reduce** complexity and risk while enabling the system to meet business needs over time.

Architectural Patterns



- Architectural patterns provide proven, reusable solutions to common system design problems.
- They define how major components are **structured** and how **responsibilities** are distributed.
- Common patterns address concerns such as scalability, maintainability, and fault tolerance.
- Choosing an architectural pattern influences technology choices and development practices.
- Well-chosen patterns help teams build systems that are easier to evolve and operate over time.

Why Architectural Patterns Matter

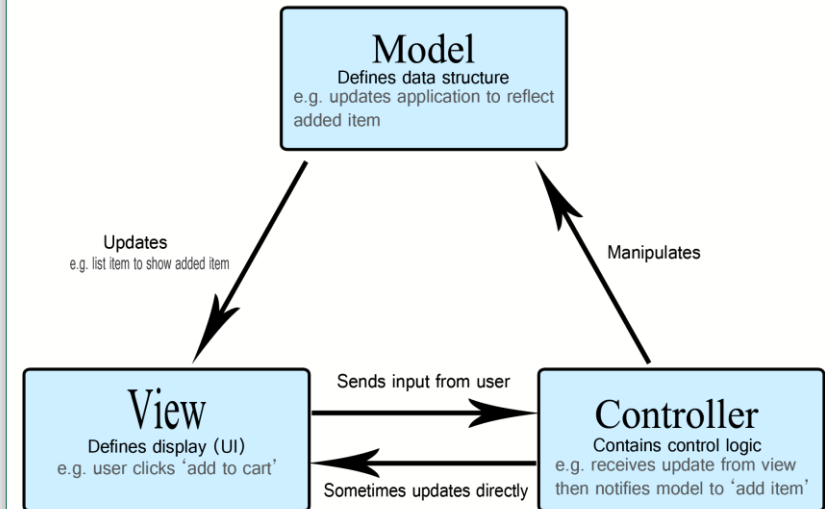


- Architectural patterns help architects avoid reinventing solutions to **common design challenges**.
- They provide a shared vocabulary for discussing system structure and trade-offs.
- Patterns make design decisions more explicit and easier to reason about.
- They guide early choices that are **difficult** and **costly** to change later.
- Using patterns improves consistency across systems and teams.

MVC



- MVC separates an application into **Model**, **View**, and **Controller** components.
- The model manages data and business logic.
- The view is responsible for presenting data to the user.
- The controller handles user input and coordinates interactions between the model and the view.
- This separation improves **maintainability**, **testability**, and clarity of **responsibility**.



Key Benefits of MVC



- MVC enforces clear **separation of concerns** within an application.
- It allows changes to the user interface without impacting business logic.
- The pattern supports **parallel development** across teams.
- It improves testability by isolating logic from presentation.
- MVC is well suited for interactive applications with user-driven workflows.

When to Use MVC?



- MVC works well for applications with frequent user interaction.
- It is effective when business logic and presentation need to evolve independently.
- The pattern suits web and desktop applications with clear UI flows.
- MVC is useful when testability and maintainability are priorities.
- It may be **less suitable** for very small or highly distributed systems.



Limitations of MVC

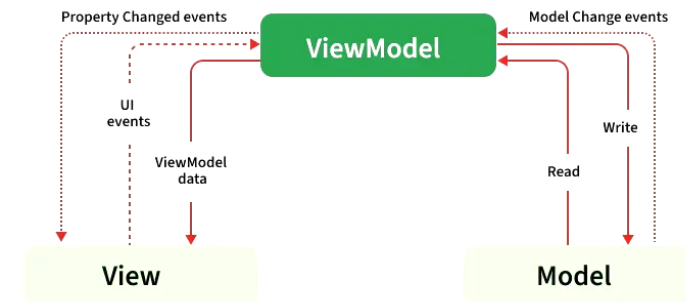


- MVC can introduce additional complexity for simple applications.
- The controller can become overly large if responsibilities are not carefully managed.
- Tight coupling may emerge if boundaries between model, view, and controller are not enforced.
- The pattern can be harder to apply in highly asynchronous or event-driven systems.
- Poor implementation can reduce the clarity that MVC is intended to provide.

MVVM



- MVVM separates the user interface from business logic using a **ViewModel layer**.
- The Model represents the domain data and core logic.
- The View is responsible only for **rendering** the user interface.
- The ViewModel exposes data and behavior in a form the view can bind to.
- This pattern promotes testability and clean separation between UI and logic.



Key Benefits of MVVM



- MVVM enables strong separation between UI and business logic.
- It improves testability by allowing ViewModels to be tested independently of the view.
- Data binding reduces boilerplate code and simplifies UI updates.
- The pattern supports parallel development of UI and logic.
- MVVM works well with modern reactive and declarative UI frameworks. Like Android (Jetpack, Compose), Vue.js, Flutter, and others.

When to Use MVVM?



- MVVM is well suited for applications with complex, data-driven user interfaces.
- It is effective when the UI requires **frequent state updates**.
- The pattern fits frameworks with strong data-binding support.
- MVVM is useful when maintainability and automated testing are priorities.
- It is commonly used in **large-scale front-end applications**.

MVVM Limitations



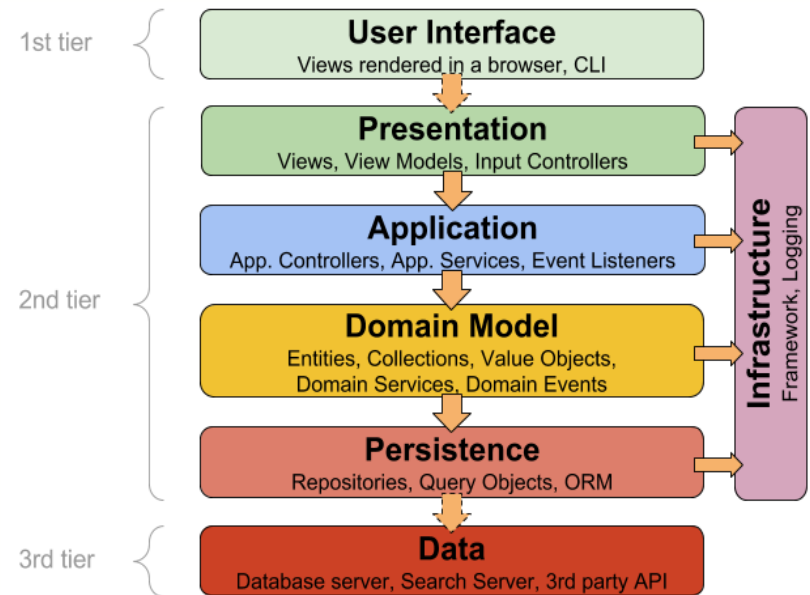
- MVVM can be harder to understand for teams new to data binding concepts.
- Debugging data-binding issues can be challenging.
- ViewModels may become overly complex if not carefully designed.
- The pattern can introduce overhead for simple applications.
- Poor binding design can lead to performance issues.



Layered Architecture



- Layered architecture structures a system into distinct layers with clear responsibilities.
- Each layer **provides services** to the **layer above** and **depends** on the **layer below**.
- A common setup includes presentation, application/service, domain, and data/infrastructure layers.
- The pattern simplifies development by keeping concerns separated and boundaries explicit.
- Layered architecture is widely used in enterprise applications where clarity and maintainability matter.



Key Benefits of Layered Architecture



- Layering improves **maintainability** by isolating changes within a single layer.
- It makes the system easier to understand by enforcing clear separation of concerns.
- It supports reuse because shared logic can live in lower layers.
- It enables **independent testing** of layers using mocks or interfaces.
- It can improve team organization by mapping teams to layers.

When to Use Layered Architecture?



- Layered architecture works well for **CRUD-heavy** business applications.
- It fits systems with stable requirements and well-understood workflows.
- It is a good choice when you need clear governance and separation between UI, logic, and data.
- It suits organizations that want consistent structure across multiple applications.
- It is often used when compliance and security **boundaries** must be explicit.

Layered Architecture Limitations

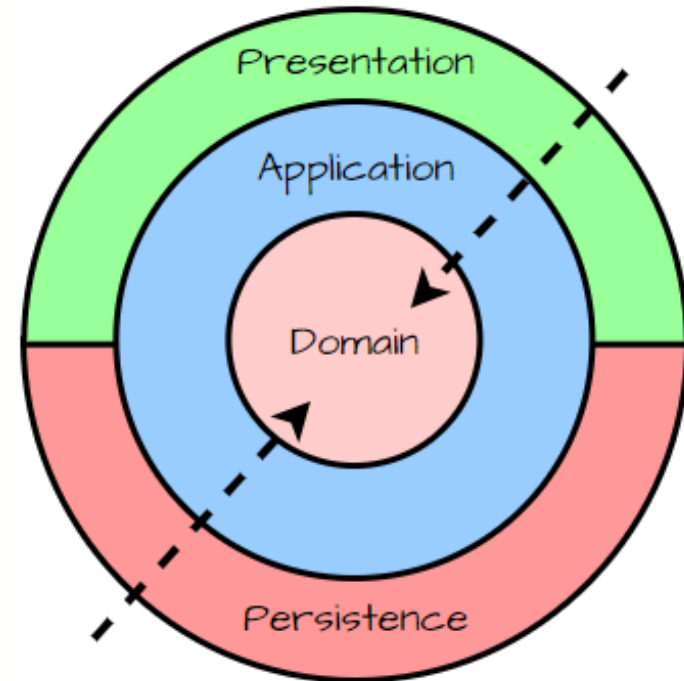


- Strict layering can add latency because requests pass through multiple layers.
- Over-layering can lead to boilerplate and slow down delivery.
- It can be **inflexible** for highly modular or rapidly evolving domains.
- Cross-cutting concerns can leak across layers if not handled consistently.
- It may not scale as well as distributed patterns for very large systems.

Clean Architecture



- Clean Architecture separates business logic from technical and infrastructure concerns.
- It organizes the system around **use cases** rather than frameworks or delivery mechanisms.
- The architecture enforces independent layers with **clear responsibility boundaries**.
- This approach improves testability, maintainability, and long-term scalability.
- It allows systems to evolve and adopt new technologies with minimal impact on core logic.



Key Benefits of Clean Architecture



- Clean Architecture improves maintainability by clearly separating responsibilities across layers.
- It increases testability by isolating business logic from frameworks and external systems.
- The approach supports **long-term scalability** as applications grow in size and complexity.
- It enables easier technology changes without impacting core business rules.

When to Use Clean Architecture?



- Clean Architecture is well suited for large or long-lived applications that are expected to evolve over time.
- It is effective in systems with **complex business rules** or multiple delivery mechanisms.
- The architecture works well for enterprise applications, microservices, and **domain-driven designs** (more on that later!).
- It is most valuable when maintainability and testability are higher priorities than rapid prototyping.

Limitations of Clean Architecture

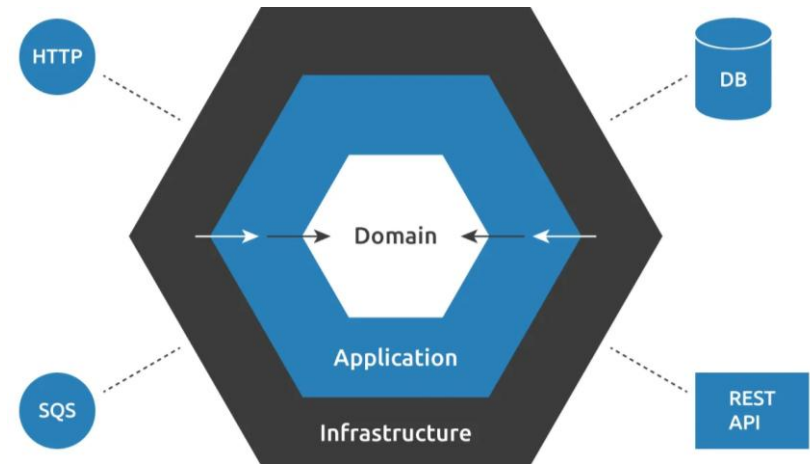


- Clean Architecture introduces **additional complexity** and requires more upfront design effort.
- It may be unnecessary or inefficient for small, short-lived, or simple applications.
- The learning curve can be steep for teams unfamiliar with architectural patterns.
- **Poor implementation** can lead to over-engineering and excessive abstraction.

Hexagonal Architecture



- Hexagonal Architecture structures applications around the **core business logic**.
- It isolates the application from external systems such as databases, UIs, and APIs.
- The architecture communicates through **ports and adapters** instead of direct dependencies.
- This design emphasizes flexibility and independence from technical details.



Key Benefits - Hexagonal Architecture



- Hexagonal Architecture improves testability by allowing **external systems** to be easily mocked or replaced.
- It enables flexibility by making it simple to **swap technologies** such as databases or user interfaces.
- The architecture promotes a clear separation between business logic and infrastructure code.
- It supports maintainable and adaptable systems over time.

When to Use Hexagonal Architecture?



- Hexagonal Architecture is well suited for applications with **multiple external integrations**.
- It works effectively in systems where business logic must remain stable while technology evolves.
- The approach is commonly used in backend services, APIs, and domain-driven designs.
- It is most beneficial in medium to large systems with long-term maintenance needs.

Limitations of Hexagonal Architecture

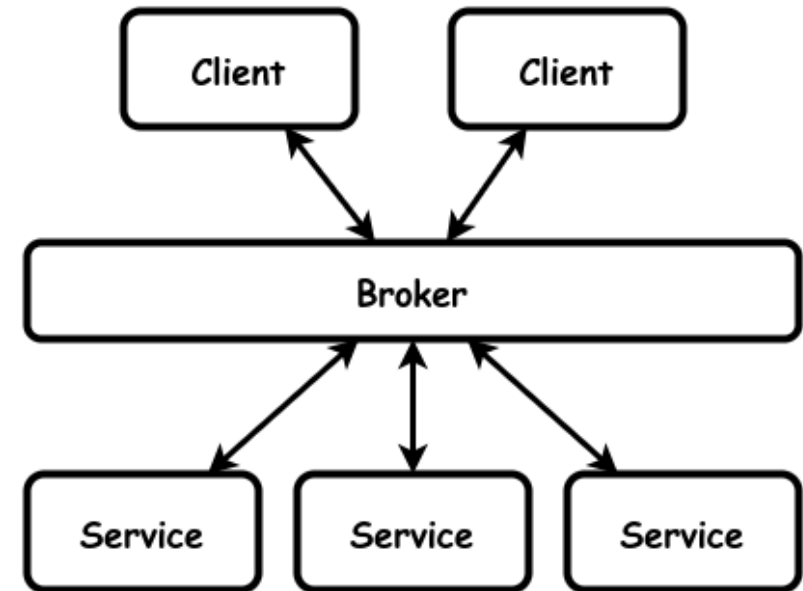


- Hexagonal Architecture can introduce additional abstraction and boilerplate code.
- It may be excessive for small or simple applications with minimal business logic.
- Teams unfamiliar with ports and adapters may face a learning curve.
- Poor design decisions can lead to unnecessary complexity and reduced clarity.

Broker Architecture



- Broker Architecture structures systems around a **central broker** that manages communication between components.
- It decouples service providers from service consumers through **message-based interaction**.
- Components **do not** communicate directly and instead rely on the broker for coordination.
- This architecture is commonly used in **distributed** and **event-driven systems**.



Key Benefits of Broker Architecture



- Broker Architecture improves **scalability** by allowing components to be added or removed independently.
- It enables **loose coupling** between services, which increases flexibility and maintainability.
- The architecture supports **asynchronous communication** and fault tolerance.
- It simplifies integration in **distributed systems** with many interacting components.

When to Use Broker Architecture?



- Broker Architecture is well suited for distributed systems and **microservices-based** environments.
- It is effective in **event-driven systems** and applications requiring **asynchronous communication**.
- The approach is commonly used in messaging systems, enterprise integration platforms, and middleware solutions.
- It is beneficial when systems must support high scalability and dynamic service discovery.

Limitations of Broker Architecture

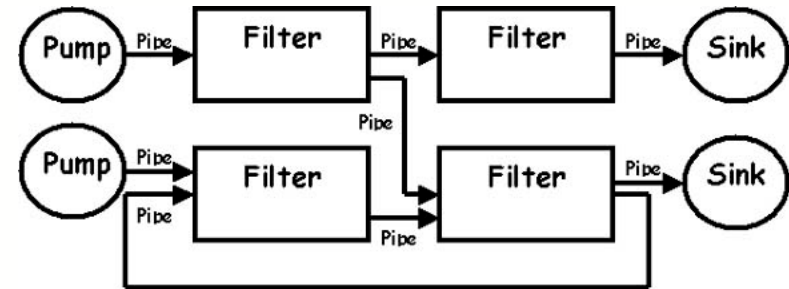


- Broker Architecture can introduce **performance overhead** due to message routing and mediation.
- **Debugging** and **tracing** issues can be more complex in asynchronous, message-driven systems.
- The broker can become a **single point of failure** if not designed for high availability.
- The architecture may be excessive for simple or tightly coupled applications.

Pipe & Filter Architecture



- Pipe and Filter Architecture structures systems as a series of **independent processing components** called filters.
- Data flows between filters through **pipes**, which act as communication channels.
- Each **filter** performs a single transformation and does not share state with others.
- This architecture emphasizes composability and **sequential data processing**.



Key Benefits of Pipe & Filter Arch.



- Pipe and Filter Architecture improves **modularity** by separating processing into independent filters.
- It enables reuse and easy replacement of individual filters without affecting the entire system.
- The architecture supports **parallel processing** and scalable data flows.
- It simplifies testing by allowing each filter to be tested in isolation.

When to Use Pipe & Filter Arch. ?



- Pipe and Filter Architecture is well suited for **data transformation, streaming,** and **batch processing** systems.
- It is effective in scenarios where data must pass through a well-defined sequence of steps.
- The approach is commonly used in compilers, ETL pipelines, and **signal processing** systems.
- It works best when processing stages are independent and stateless.

Limitations of Pipe & Filter Arch.

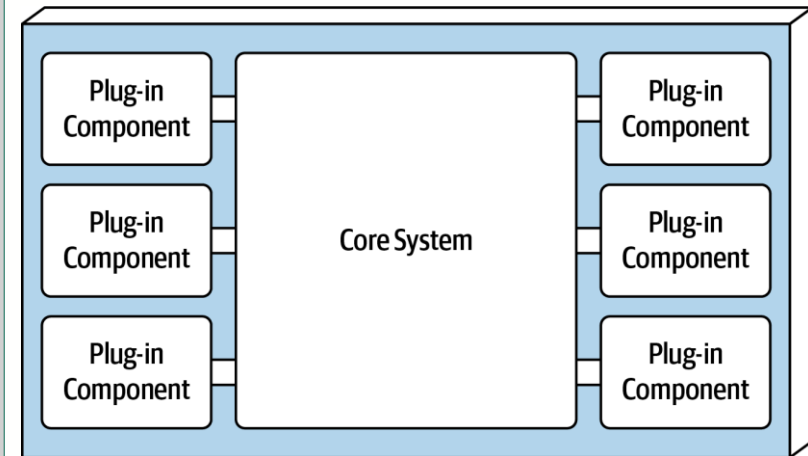


- Pipe and Filter Architecture can be inefficient when filters require shared context or state.
- **Error handling** and debugging can be challenging across multiple filters.
- The architecture may introduce **latency** due to sequential data flow.
- It is less suitable for interactive systems or workflows with complex branching logic.

Microkernel Architecture



- Microkernel Architecture structures systems around a minimal core that provides essential functionality.
- Additional features are implemented as **plug-in modules** that extend the core behavior.
- The **core** remains stable while extensions can be added, removed, or replaced independently.
- This architecture emphasizes **extensibility** and long-term adaptability.



Key Benefits of Microkernel Arch.



- Microkernel Architecture enables high extensibility through well-defined plug-in mechanisms.
- It improves maintainability by keeping the **core** system **small** and **stable**.
- The architecture allows new functionality to be added without modifying the core.
- It supports product lines and systems with varying feature sets.

When to Use Microkernel Arch. ?



- Microkernel Architecture is well suited for applications that require frequent customization or **extensions**.
- It is effective in systems that support multiple deployment configurations or **customer-specific features**.
- The approach is commonly used in **IDEs**, operating systems, and plugin-based platforms.
- It works best when core functionality is clearly defined and stable.

Limitations of Microkernel Arch.



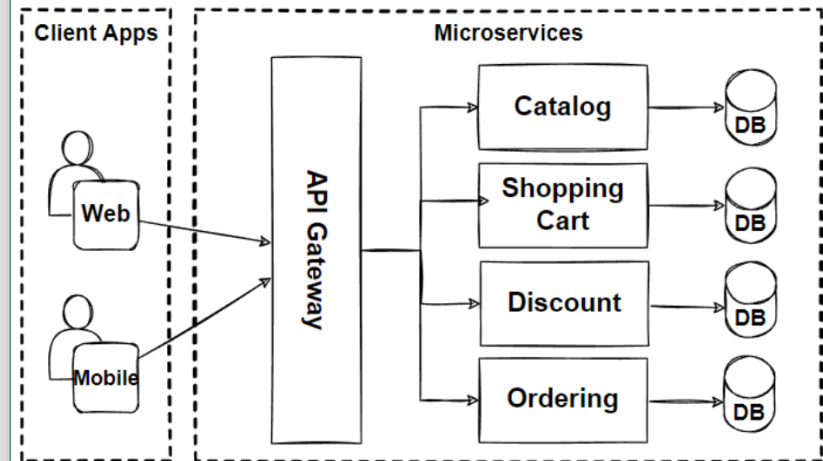
- Microkernel Architecture can introduce complexity in plugin management and communication.
- Performance overhead may occur due to indirection between the core and extensions.
- Designing stable and flexible **extension points** requires careful upfront planning.
- The architecture may be excessive for simple applications with limited variability.



Microservice Architecture



- Microservices Architecture structures applications as a collection of small, **independent services**.
- Each service is responsible for a specific business capability and can be developed and deployed independently.
- Services communicate with each other through lightweight APIs or messaging.
- This architecture emphasizes **scalability**, **autonomy**, and **continuous delivery**.



Key Benefits of Microservice Arch.



- Microservices Architecture enables **independent** development and deployment of services.
- It improves scalability by allowing services to scale based on individual demand.
- The architecture increases system resilience by isolating failures to individual services.
- It supports faster innovation and continuous delivery cycles.

When to Use Microservice Arch. ?



- Microservices Architecture is well suited for **large, complex, and rapidly evolving** systems.
- It is effective in organizations with **multiple development teams** working in parallel.
- The approach is commonly used in cloud-native and highly scalable applications.
- It works best when systems require frequent updates and independent scaling.

Limitations of Microservice Arch.



- Microservices Architecture introduces operational complexity, including service orchestration and monitoring.
- Network communication can increase latency and failure scenarios.
- Data consistency becomes more challenging across distributed services.
- The architecture may be excessive for small applications or teams.